

Topic 8.3

8.3: Exceptions

Unfortunately, *setjmp* and *longjmp* are incompatible with many C++ features, such as automatically invoking destructors for objects when they go out of scope. Thus C++ uses *exceptions* to perform error handling.

We first define exceptions, which are really just classes themselves, to indicate the types of errors we will deal with.

When we want to write a section of code in which errors are to be detected and handled, we use a *try* block, which provides a special mechanism for detecting errors. The *try* block is followed by one or more *catch* blocks, each of which handles one or more possible errors that could occur in the *try* block.

When we write a section of code and discover that some error has actually occurred, we *throw* the proper exception. This causes the runtime stack to unwind to the nearest point at which the exception is handled.

The following program demonstrates how to use exceptions:

```
// PROGRAM:      mpgexcep.cpp
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        97/05/12
//
// PURPOSE:
//   Calculate gas mileage.
//   Illustrate use of exceptions
//   for error handling.

#include <iostream.h>

class InitOdomErr {};
class FinalOdomErr {};
class GasErr{};

void read_inputs( float& init_odom,
                  float& final_odom,
                  float& gas );
float mileage( float init_odom,
               float final_odom,
               float gas );
void print_result(float mpg);

void read_inputs( float& init_odom,
                  float& final_odom,
                  float& gas )
{
    cout << "Enter initial odometer reading ";
    cin >> init_odom;
    cout << "Enter final odometer reading  ";
    cin >> final_odom;
    cout << "Enter gallons of gas used      ";
```

```

    cin >> gas;
}

float mileage( float init_odom, float final_odom,
               float gas )
{
    if (init_odom < 0)
        throw InitOdomErr();
    if (final_odom < init_odom)
        throw FinalOdomErr();
    if (gas <= 0)
        throw GasErr();
    return (final_odom - init_odom) / gas;
}

void print_result(float mpg)
{
    cout << "Your gas mileage is " << mpg << endl;
}

void main(void)
{
    int success = 0;           // Computation
                                // successful?
    float initial_odometer;    // Initial odometer
                                // reading in miles.
    float final_odometer;      // Final odometer
                                // reading in miles.
    float gas_used;            // Gallons of gas
                                // used.
    float mpg;                 // Miles per gallon.

    while (! success)
    {
        try
        {
            read_inputs (initial_odometer,
                          final_odometer,
                          gas_used);
            mpg = mileage (initial_odometer,
                          final_odometer,
                          gas_used);
            print_result(mpg);
            success = 1;
        }
        catch (InitOdomErr)
        {
            cout <<
                "*** Error in initial
                odometer reading.\n";
        }
        catch (FinalOdomErr)
        {
            cout <<
                "*** Error in final
                odometer reading.\n";
        }
        catch (GasErr)
        {
            cout <<
                "*** Error in amount of gas used.\n";
        }
    }
}

```

Output from mpgexcep.cpp:

```

Enter initial odometer reading 1000
Enter final odometer reading 900
Enter gallons of gas used 12
*** Error in final odometer reading.
Enter initial odometer reading 1000
Enter final odometer reading 1200
Enter gallons of gas used -3
*** Error in amount of gas used.
Enter initial odometer reading 1000
Enter final odometer reading 1240
Enter gallons of gas used 8
Your gas mileage is 30

```

The following program illustrates the declaration of exceptions within classes that they are associated with.

```

// MODULE:      dynarr2d.cpp
// PROGRAMMER:   Gary J. Blair
// LANGUAGE:     C++
// DATE:        96/04/11
//
// PURPOSE:
//   Driver for dynarr2 exercise.
//   Illustrate exceptions.

#include <iostream.h>
#include <stdlib.h>
#include <string.h>

class DynamicIntArray
{
public:
    DynamicIntArray( size_t size,
                    const char * const name );
    ~DynamicIntArray(void);
    int& operator[]( long subscript );
    size_t size(void);

class SubscriptError {}; // Exception class.

private:
    size_t array_size;      // Size of the
                          // dynamic array.
    char array_name[80];    // For educational
                          // purposes only.
    int *data;              // Pointer to
                          // dynamic
                          // memory for the
                          // array
};

// Constructor
DynamicIntArray::DynamicIntArray
( size_t size, const char * const name )
{
    long i; // Loop variable.

    cout << "constructor called for " << name
          << endl;

```

```

    data = new int[size];
    if (! data)
    {
        cerr << "**** Memory allocation failed.\n";
        exit(1);
    }
    array_size = size;
    for (i=0; i<array_size; ++i)
        data[i] = 0;
    strcpy(array_name,name);
}

// Destructor.

DynamicIntArray::~DynamicIntArray(void)
{
    delete data;
    array_size = 0;
    cout << "Destructor called for " << array_name
        << endl;
}

// Accessor function which returns
// a reference to an element of
// the dynamically-allocated memory.

int& DynamicIntArray::operator[]( long  subscript )
{
    if (subscript < 0 || subscript >
        (array_size - 1))
        throw SubscriptError();
    return data[subscript];
}

// Inquire the size of the array.

size_t DynamicIntArray::size(void)
{
    return array_size;
}

void main(void)
{
    DynamicIntArray  x(23,"x");
    long            i;

    for (i=0; i<23; ++i)
        x[i] = i * i;

    try
    {
        cout << "x[13]=" << x[13] << endl;
        cout << "x[27]=" << x[27] << endl;
    }
    catch (DynamicIntArray::SubscriptError)
    {
        cout <<
            "**** Subscript error accessing array.\n";
    }
}

```

Output from dynarr2d.cpp:

```
constructor called for x  
x[13]=169  
*** Subscript error accessing array.  
Destructor called for x
```

Topic 8.4 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-8-dot-4>)